

# Using HLS IP in a Zynq SoC Design

## Overview

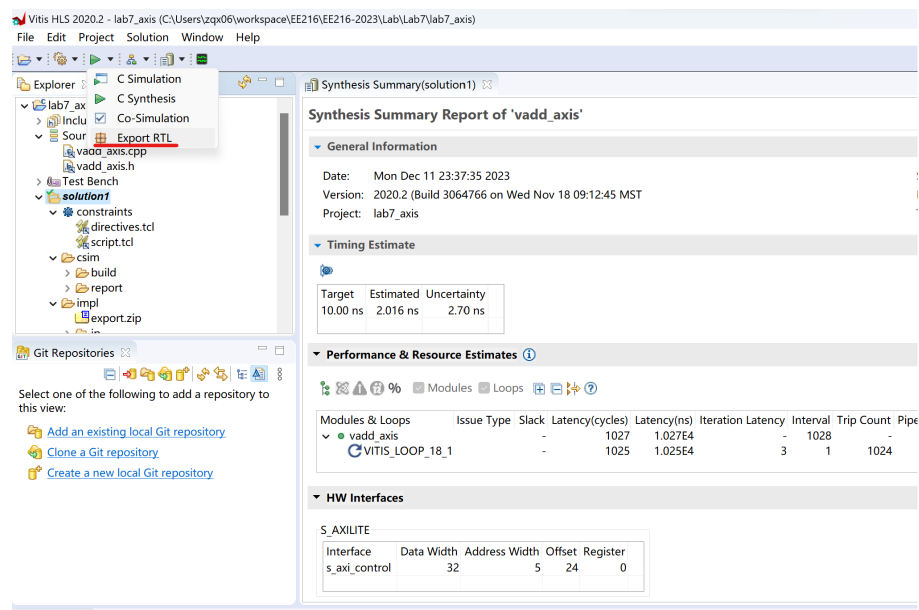
A common use of High-Level Synthesis design is to offload some tasks from CPU to FPGA by creating an accelerator that can run faster on FPGAs. This tutorial shows you how to create a design that make CPU and FPGA collaborate on a Zynq device.

You will create and configure a simple HLS design to work with the CPU on a Zynq device. The HLS design used in this lab is simple to allow the focus of the tutorial to be on explaining the connections to the CPU and how to configure the software drivers created by High-Level Synthesis to control the device and manage interrupts.

## Steps

### Step 1: Create a Vivado HLS IP Block

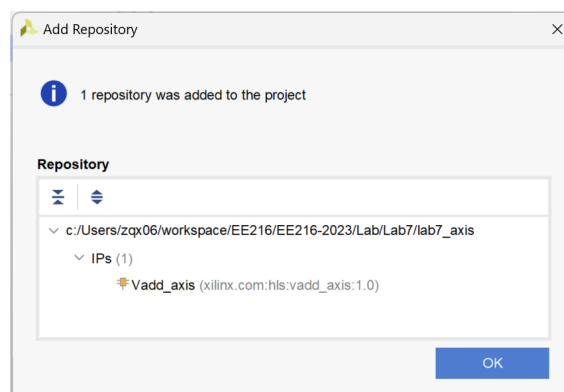
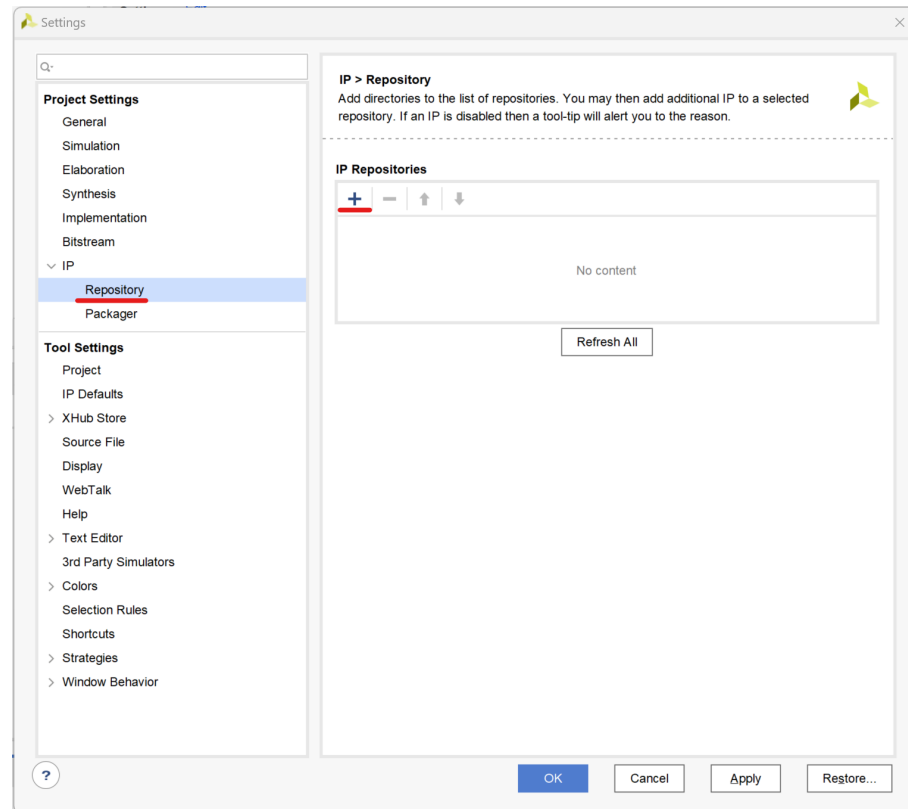
1. Launch **Vitis HLS**
2. Create a project with name **lab7\_hls**
3. Add the provided source file **vadd\_axis.cpp** and **vadd\_axis.h**. Open and have a look on the source code, specially for the INTERFACE pargmas.
4. Add the testbench file **vadd\_axis\_tb.cpp**
5. (Optional) Run **C Simulation**
6. Run **C Synthesis**
7. (Optional) Run **Co Simulation**
8. After synthesis, package the IP by clicking on **Export RTL**.



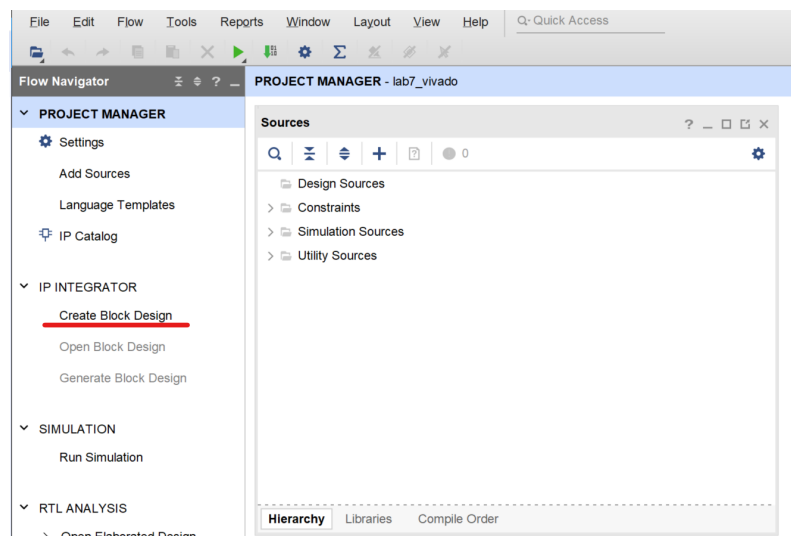
The remainder of this lab shows **how the HLS IP blocks can be integrated into a Zynq block design using Vivado IP integrator**.

## Step 2: Create Block Design in Vivado

1. Launch **Vivado**, create a new project with name **lab7\_vivado**, select **ZCU104** as the target board.
2. Click **Settings** from the **Porject Manager**
3. Select **IP > Repository**, Click the **Add** button, browse to select the **lab7\_hls** folder, click **OK**.  
You should see a information that the new created IP is detected.

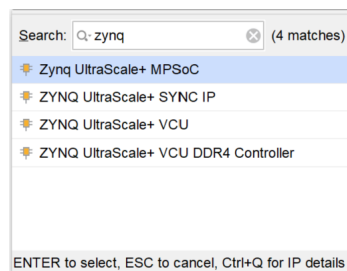


4. Click **Apply** and **OK** to close the Settings window.
5. From the **IP Integrater**, click **Create Block Design**. Click **OK**.

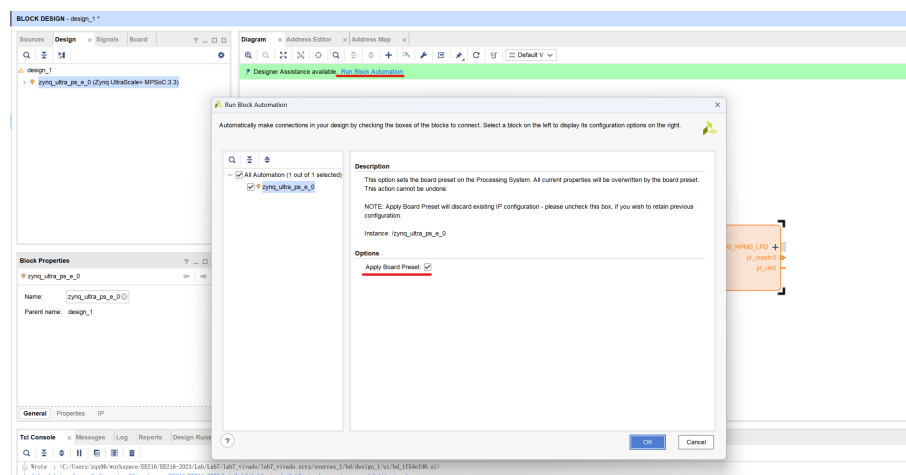


6. Click the **Add** button to add an IP.

7. Search for **zynq**, and add the **Zynq UltraScale+ MPSoC** to the block design by double clicking on it.



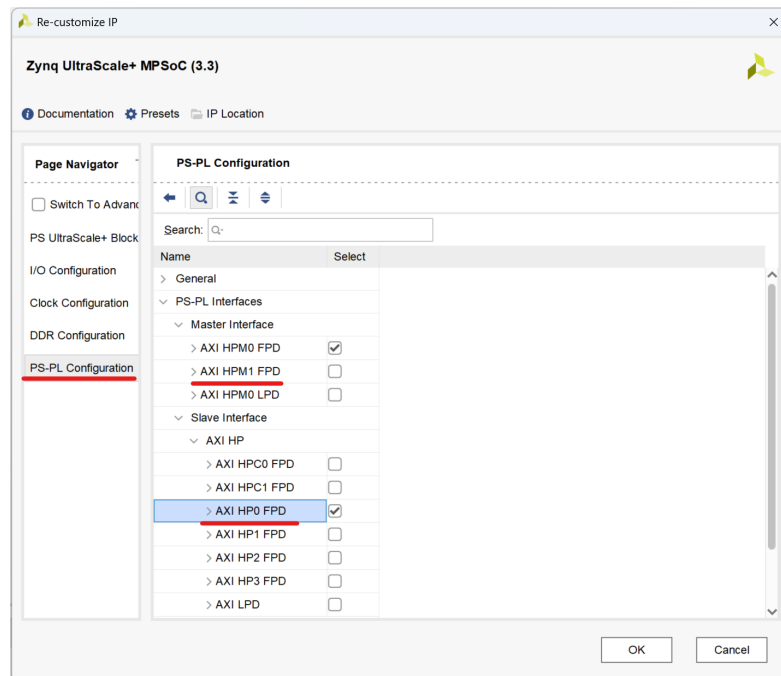
8. Click **Run Block Automation**, select **Apply Board Preset**, click **OK**.



9. Double-clicking on the new-added block **zynq\_ultra\_ps\_e\_0** to re-customize it.

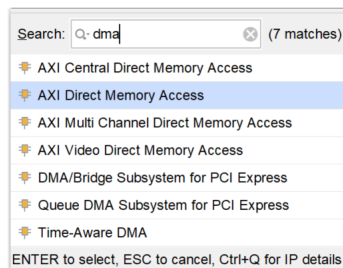
10. Select the tag **PS-PL Configuration**, Expand **PS-PL Interfaces > Master Interface**, uncheck the box **AXI HPM1 FPD**.

11. Expand **PS-PL Interfaces > Slave Interface**, check the box **AXI HP0 FPD**.

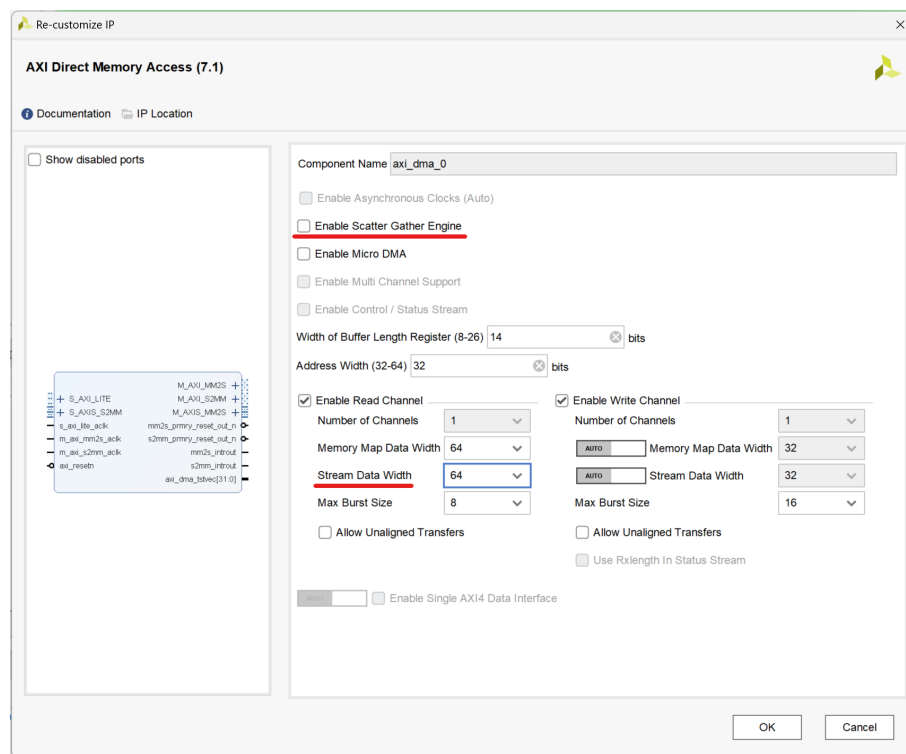


12. Click **OK** to close the Re-customize IP window.

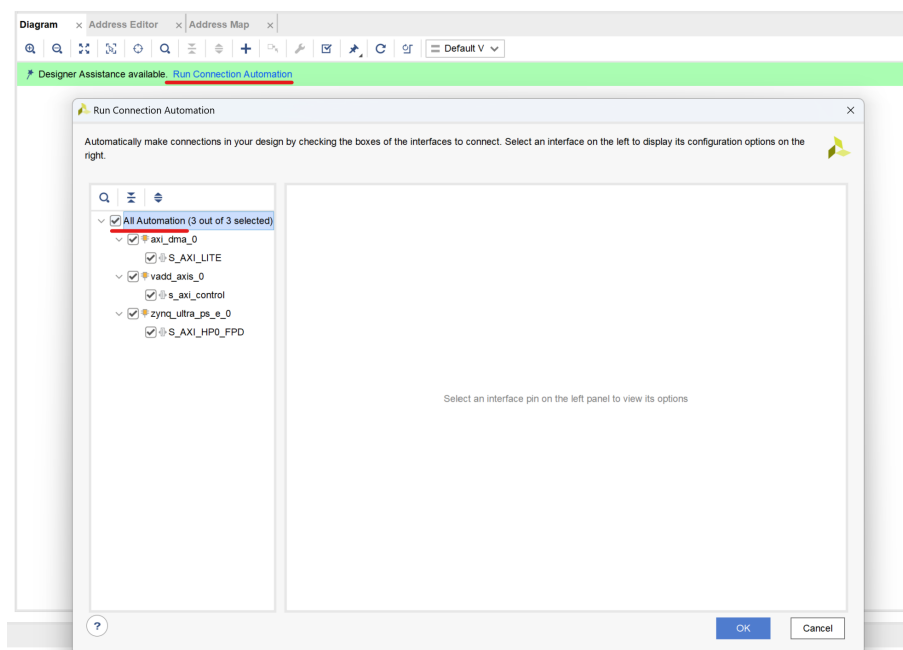
13. Search to add a **AXI Direct Memory Access** and the HLS IP **Vadd\_axis**.



1. Double-clicking on the **axi\_dma\_0** block to re-customize the DMA. Uncheck the **Enable Scatter Gather Engine**, change the **Stream Data Width** of the Read Channel to **64**. Click **OK** to close the window.



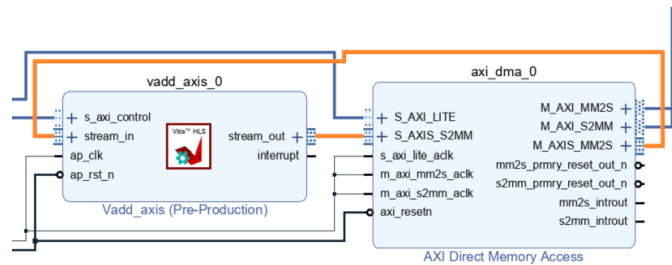
2. Click **Run Connect Automation**, select **All Automation**, click **OK**.



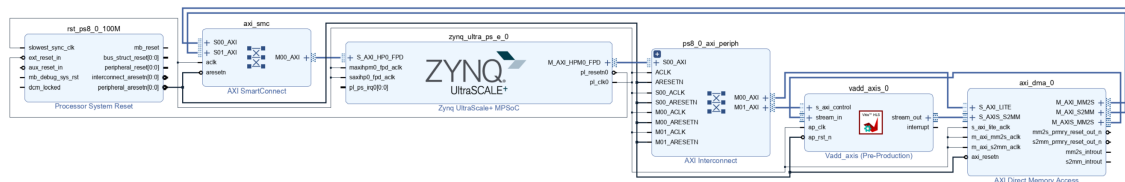
3. Click **Run Connect Automation** again, select **All Automation**, click **OK**.

4. Using your mouse to create two connections (Left-clicking on one port and drag it to another):

- axi\_dma\_0/M\_AXIS\_MM2S > vadd\_axis\_0/stream\_in
- vadd\_axis\_0/stream\_out > axi\_dma\_0/S\_AXIS\_S2MM

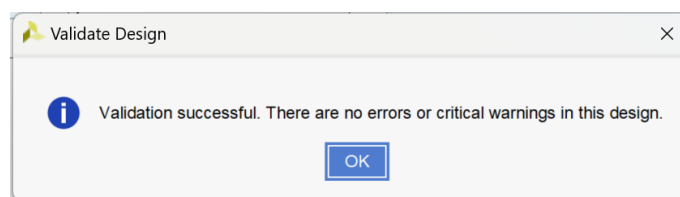
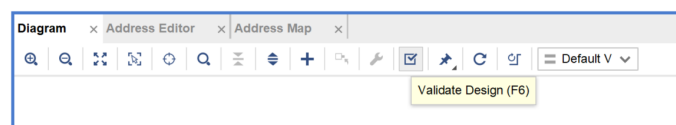


5. Click the **Regenerate Layout** button, the final block design should be like:



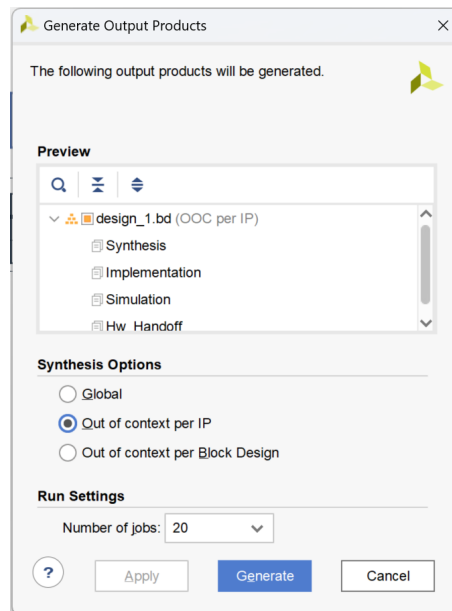
## Step3: Generate bitstream

1. Click the **Validate Design** button, make sure the block design has no errors. Click **OK**.

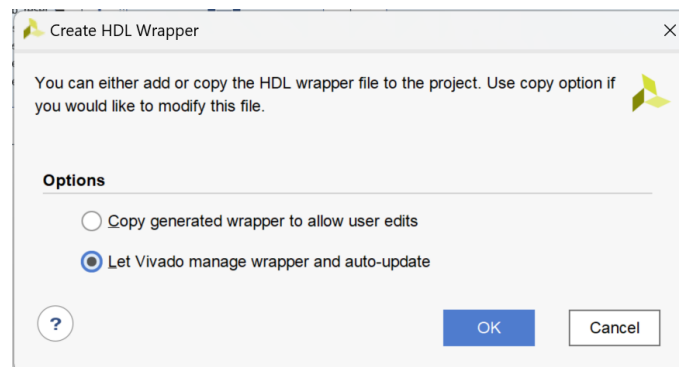
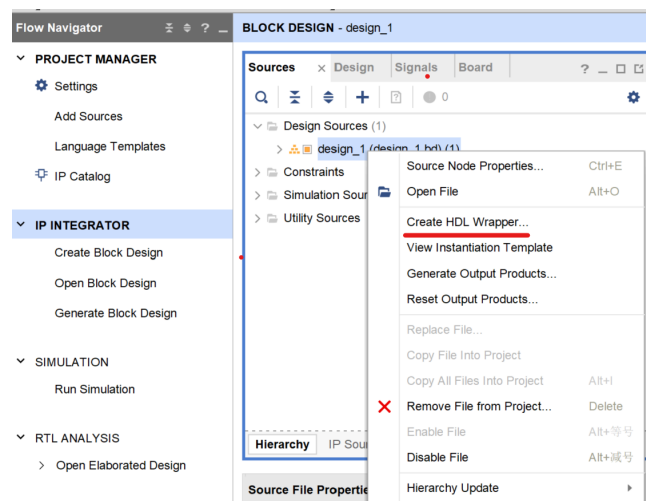


2. Save the block design using **Ctrl+S** or the **Save** button.

3. From the **IP Integrator**, click **Generate Block Design**, Change the number of jobs according to your memory and CPU cores. Click **Generate**.



4. Waiting for the process to complete. From the **Sources** tab, right-clicking on the **design\_1** (**design\_1.bd**) and select the **Create HDL Wrapper**. Select **Let Vivado manage...** and click **OK**.



5. Click **Generate Bitstream**. Click **OK** for any popped dialog windows.

- When finished, export the **bitstream file** and the **hardware handoff** with name **lab7.bit** and **lab7.hwh**. You can refer to previous labs for how to export these files.

## Step 4: Write software driver for the IP

In this lab, we will use the PYNQ framework to drive our HLS IP.

- Upload the bitstream and HWH to the board. Create a python script with name **lab7.py**
- Edit the script as bellow:

```
from pynq import Overlay, allocate
import numpy as np

# Download bitstream
overlay = Overlay("lab7.bit")

# Get the IP object from the overlay
vadd = overlay.vadd_axis_0
dma = overlay.axi_dma_0

# Allocate buffers
length = 128
in_buffer = allocate(shape=(length,), dtype=np.uint64)
out_buffer = allocate(shape=(length,), dtype=np.int32)

# Initialize input data
for i in range(length):
    in_buffer[i] = i | (i << 32)

# Launch the IP and DMA
# The registers can be seen in solution1/impl/ip/drivers/*/src/*_hw.c
vadd.write(0x18, length)
vadd.write(0x0, 0x1)
dma.sendchannel.transfer(in_buffer)
dma.recvchannel.transfer(out_buffer)
dma.sendchannel.wait()
dma.recvchannel.wait()

# Print the output
print(out_buffer)
```

- Run the script by `python3 lab7.py`

## Summary

In this lab:

- You generated a IP with AXI-Stream interface using Vitis HLS.
- You have learned how to create a zynq block design in Vivado to integrate the HLS IP and AXI DMA.
- You have learned how to write software code to drive the FPGA using PYNQ.